

DOCKER

The Complete Beginner-to-Confident Course

A personal teacher, textbook, workbook, and practice system in one — built to take you from “I’ve never used Docker” to “I can containerize, run, and ship a real backend application with confidence.”

Written for beginners — no prior DevOps experience assumed.

How To Use This Course

This is not a book to read passively from start to finish. It is a workbook. The goal is not “I finished the PDF” — the goal is “I can figure Docker problems out myself.”

The Learning Loop

Use this loop for every lesson:

- 1 **Learn** — Read the lesson once, slowly.
- 2 **Recall** — Close the PDF and try to summarize what you just read, out loud or on paper.
- 3 **Practice** — Actually type the commands and code. Do not just read them.
- 4 **Explain** — Answer the “Can You Explain It?” prompt at the end of the lesson.
- 5 **Review** — When you hit a Review Session, test yourself on old material, not just the newest lesson.
- 6 **Apply** — Build the projects. This is where real understanding forms.

■ STOP AND DO THIS

Before you continue past this page, open a terminal. You will need Docker installed (Docker Desktop on Windows/Mac, or Docker Engine on Linux) by the time you reach Lesson 3.1. You don't need it installed yet — just know it's coming.

How This PDF Is Organized

- **Levels 0–7** — a structured path from prerequisites to a final independent project.
- **Lessons** — each follows the same 13-part teaching structure so you always know what to expect.
- **Review Sessions** — mixed old + new questions, appearing periodically.
- **Knowledge Checkpoints** — a self-check gate before you move to the next level.
- **Projects** — hands-on builds that combine several lessons at once.
- **Final Exam, Checklist, Glossary, Cheat Sheet, and Next Steps** — at the end of the course.

Icon Legend

● Important	A core idea worth re-reading.
◆ Tip	A shortcut or good habit.
■ Common Mistake	Something beginners typically get wrong.
◆ Think About This	A moment to pause and reflect before continuing.
🔗 Practice	Something you should actually do, not just read.
■ Stop And Do This	A hard stop — complete the task before reading on.

Learning Progress Tracker

Fill this in as you go. Confidence scale: 1 = I don't understand it yet → 5 = I could teach this to someone else.

#	Topic	Status	Confidence (1-5)
1.1	What Is Docker and Why It Exists	■	_____
1.2	Containers vs Virtual Machines	■	_____
1.3	Images vs Containers	■	_____
2.1	The Dockerfile	■	_____
2.2	Docker CLI Essentials	■	_____
2.3	Volumes and Persistent Data	■	_____
3.1	Building Your First Image	■	_____
3.2	Docker Compose Basics	■	_____
5.1	Networking Between Containers	■	_____
5.2	Environment Variables and Secrets	■	_____
5.3	Multi-Stage Builds	■	_____
P1	Mini Project: Dockerize a FastAPI App	■	_____
P2	Mini Project: FastAPI + Postgres with Compose	■	_____
P3	Final Project: Production-Style Compose Stack	■	_____

Course Roadmap

Docker doesn't need every level a huge subject would need, so this roadmap is streamlined: it merges pure exercises into the projects, and merges scenario problem-solving directly into Level 5 and the review sessions, rather than giving it an isolated level. This teaches the subject better than forcing eight rigid levels.

Level	Focus	What You'll Be Able To Do After
0	Prerequisites	Understand what a terminal, OS, and "process" are.
1	Foundations	Explain what Docker is and why it exists, in plain words.
2	Core Concepts	Read a Dockerfile, use core CLI commands, understand volumes.
3	Practical Usage	Build and run your own image; use Docker Compose.
4	Problem Solving	Debug common container failures independently (woven into Review Sessions and Projects).
5	Intermediate Concepts	Connect containers over a network, manage secrets, write multi-stage builds.
6	Mini Projects	Dockerize a real FastAPI backend, alone and with a database.
7	Final Challenge	Design a production-style multi-service stack from a blank folder.

◆ GOAL

By the end of this course you will be able to say: "I understand what Docker is, why it exists, how it works, when I should use it, and I can explain it myself — and I can containerize a real backend project."

Table of Contents

How To Use This Course	2
The Learning Loop	2
How This PDF Is Organized	2
Icon Legend	2
Learning Progress Tracker	3
Course Roadmap	4
Level 0 — Prerequisites	7
What You Should Understand First	7
Level 1 — Foundations	8
Lesson 1.1 — What Is Docker, and Why Does It Exist?	8
Lesson 1.2 — Containers vs. Virtual Machines	11
Lesson 1.3 — Images vs. Containers	14
Level 2 — Core Concepts	17
Lesson 2.1 — The Dockerfile	17
Lesson 2.2 — Docker CLI Essentials	21
Lesson 2.3 — Volumes and Persistent Data	24
Review Session 1	27
Knowledge Checkpoint 1: Foundations and Core Concepts	28
Level 3 — Practical Usage	29
Lesson 3.1 — Building Your First Real Image	29
Lesson 3.2 — Docker Compose Basics	32
Level 5 — Intermediate Concepts	35
Lesson 5.1 — Networking Between Containers	35
Lesson 5.2 — Environment Variables and Secrets	38
Lesson 5.3 — Multi-Stage Builds	41
Review Session 2	44
Knowledge Checkpoint 2: Practical Usage and Intermediate Concepts	45
Level 6 — Mini Projects	46
Project — Dockerize a FastAPI Backend (Guided)	46

Project — FastAPI + PostgreSQL with Docker Compose (Semi-Guided)	48
Level 7 — Final Challenge	51
Project — Production-Style Multi-Service Stack (Independent)	51
Final Exam	54
Part 1 — Concept Questions	54
Part 2 — Explain In Your Own Words	54
Part 3 — Practical Problems	54
Part 4 — Scenario Questions	54
Part 5 — Troubleshooting	54
Part 6 — Final Project	54
Final Exam — Answer Key	56
Part 1	56
Part 2	56
Part 3	56
Part 4	57
Part 5	57
Part 6	57
Final “Can I Actually Do This?” Checklist	58
Complete Glossary	59
Cheat Sheet	61
Core Commands	61
Minimal Dockerfile Pattern	61
Minimal Compose Pattern	61
Rules of Thumb	62
What To Learn Next	63
If You Enjoyed This Subject	63
If You Want To Become Job-Ready	63
If You Want To Specialize	63
If You Want To Go Deeper	63

Level 0 — Prerequisites

Before touching Docker itself, you need three small ideas fixed in your head. If you already know them, skim this page in 60 seconds and move on.

What You Should Understand First

A Program

A program is a set of instructions saved as a file (like **python app.py** or **node server.js**). By itself, a program does nothing — it needs to be *run*.

A Process

A process is a program *while it is running*. When you start a program, your operating system (OS) gives it its own slice of memory and CPU time to work with.

An Operating System (OS)

The OS (Windows, macOS, Linux) is the software that manages your computer's hardware and runs your programs. Different programs often need different versions of system libraries to run correctly — and this is exactly the problem Docker was built to solve, which you'll see in Lesson 1.1.

A Terminal / Command Line

A terminal is a text window where you type commands instead of clicking buttons. Docker is primarily used through the terminal. If you can already run a command like **python --version** or **node --version** in a terminal, you have everything you need.

◆ TIP

If terminals are brand new to you: you don't need to be an expert. You only need to know how to open one and type a line of text followed by Enter. Everything else, this course teaches.

Level 1 — Foundations

Lesson 1.1 — What Is Docker, and Why Does It Exist?

Difficulty: BEGINNER

1. What You Will Learn

- Explain the “it works on my machine” problem in your own words.
- Define what Docker actually is, without buzzwords.
- Name the core problem Docker solves.
- Recognize a real situation where Docker would help.

2. Why This Matters

Every developer eventually hits this: code that runs perfectly on your laptop breaks the moment it runs on a teammate's machine, a server, or in production. Docker exists to make that problem disappear.

3. The Simple Explanation

Imagine you bake a cake in your own kitchen, with your own oven, your own exact ingredients, at your own altitude. It comes out perfect. Now imagine mailing the instructions to a friend in another city and asking them to reproduce it exactly. Their oven runs hotter, their flour is a different brand, their altitude is different — the cake comes out different, or fails entirely.

Docker solves this by packaging not just the recipe (your code), but the entire kitchen (the operating system pieces, libraries, and exact versions your code needs) into one sealed box called a **container**. Ship the box anywhere, and it behaves identically.

4. Key Terms

Term	Simple Definition	Example
Docker	A tool that packages an application together with everything it needs to run, so it behaves the same everywhere.	A FastAPI app + Python 3.12 + its exact dependencies, bundled into one unit.
Container	A running, isolated instance of a packaged application.	Your FastAPI app actually running, sealed off from the host machine's own setup.
“It works on my machine”	A famous developer problem: code behaves differently across environments.	A teammate's app crashes because they have Python 3.9 installed, but you built it with 3.12.

Term	Simple Definition	Example
Environment	The specific combination of OS, language version, and libraries a program needs.	Python 3.12 + FastAPI 0.115 + Ubuntu 22.04.

5. How It Works

- 1 You write application code on your machine, which has its own installed language versions and libraries.
- 2 Without Docker, you'd have to manually tell every teammate and every server exactly what to install, in what order, at what versions.
- 3 With Docker, you write one file (a **Dockerfile**, covered in Lesson 2.1) describing the environment.
- 4 Docker turns that description into an **image** — a reusable snapshot.
- 5 Anyone with Docker installed can run that image and get an identical environment, instantly.

6. Real-World Example

Companies like Netflix, Spotify, and thousands of smaller engineering teams use Docker to make sure the code that passes tests on a developer's laptop is the exact same code, in the exact same environment, that runs in production. It's also standard for local development: instead of installing PostgreSQL, Redis, and five other tools directly on your laptop, you can run each in its own container.

7. Practical Example

You don't need to install anything yet for this first lesson — just look at the shape of the idea. Here is what a teammate would type to get your entire app running, with zero manual setup, once it's dockerized:

```
docker run your-fastapi-app
```

That one line replaces: install Python, install the right version, create a virtual environment, install every dependency, set environment variables, and hope nothing conflicts with what's already on their machine.

8. Common Mistakes

■ COMMON MISTAKE

Thinking Docker is a virtual machine. It's related but different — Lesson 1.2 explains exactly how.

■ COMMON MISTAKE

Thinking you need Docker for every project. Small scripts with no dependencies often don't need it.

9. "What If?" Variations

- What if your app only depends on Python's standard library, nothing else? Docker still helps if you need the exact same Python *version* everywhere, but the benefit is smaller.
- What if you're the only person who will ever run this code, forever, on this one machine? Docker's main benefit (consistency across machines) matters less — though reproducibility for your future self still counts.

10. Check Your Understanding

- 1 In your own words, what problem does Docker solve?
- 2 What is the difference between a program and a container?
- 3 Give an example (not from this lesson) of when “it works on my machine” could happen.
- 4 True or false: Docker packages only your code, not its dependencies. Explain your answer.

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Write a two-sentence explanation of Docker for a non-technical friend, using an analogy that is NOT the cake analogy used in this lesson.

12. Lesson Summary

- Docker packages an application with everything it needs to run.
- This solves the “it works on my machine” problem.
- A container is a running instance of a packaged application.
- Docker matters most when consistency across machines or teams matters.

♦ THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- What is Docker, in one sentence, with no jargon?
- Why would a team of five developers want to use it?
- What is the difference between a program and a container, in your own words?

Lesson 1.2 — Containers vs. Virtual Machines

Difficulty: BEGINNER

1. What You Will Learn

- Explain how a container differs from a virtual machine (VM).
- Describe why containers are lighter and faster than VMs.
- Identify when you'd still choose a VM over a container.

2. Why This Matters

This is the single most common beginner confusion. Getting it right early prevents months of fuzzy mental models later.

3. The Simple Explanation

A **virtual machine** is like building an entire separate house inside your house — complete with its own foundation, walls, plumbing, and a full second operating system. It works, but it's slow to build and uses a lot of space.

A **container** is more like a room with a locked door inside your existing house. It shares the same foundation and plumbing (your computer's OS kernel) but is sealed off enough that what happens inside doesn't affect the rest of the house. It's much faster to set up and uses far fewer resources, because it isn't rebuilding an entire second house.

4. Key Terms

Term	Simple Definition	Example
Virtual Machine (VM)	Software that emulates an entire separate computer, including its own OS.	Running a full Ubuntu VM inside your Windows laptop using VirtualBox.
Kernel	The core part of an operating system that talks directly to the hardware.	Linux containers on your machine share your machine's Linux kernel.
Hypervisor	Software that creates and manages virtual machines.	VirtualBox, VMware, or Hyper-V.
Isolation	Keeping something separate so it can't interfere with (or be interfered with by) its surroundings.	A container's files are isolated from your host machine's files by default.

5. How It Works

- 1 A VM virtualizes hardware: it runs a full guest OS on top of a hypervisor, which sits on your real (host) OS.

- 2 Booting a VM means booting an entire second operating system — this can take a minute or more, and uses gigabytes of disk space.
- 3 A container, by contrast, shares your host machine's OS kernel directly.
- 4 It only packages the application layer: your code, libraries, and settings — not a second OS.
- 5 This means containers start in milliseconds to a few seconds, and are typically megabytes, not gigabytes.

6. Real-World Example

Cloud providers like AWS, Google Cloud, and Azure use VMs to give customers fully isolated servers. But once a company has its own server (VM or physical), they very often run many containers on top of it — for example, running 10 different microservices in 10 lightweight containers on a single VM, instead of running 10 full separate VMs, which would be slow and expensive.

7. Practical Example

Here's the resource difference made concrete. This is illustrative, not something you need to run:

```
Virtual Machine: ~2-20 GB disk, ~30-60 sec to boot, full guest OS
Container:       ~50-500 MB disk, ~1-2 sec to start, shares host OS kernel
```

8. Common Mistakes

■ COMMON MISTAKE

Saying “a container is a lightweight VM.” It's architecturally different — it shares the host kernel rather than virtualizing hardware.

■ COMMON MISTAKE

Assuming containers are less secure than VMs by default because they share a kernel. In practice, container isolation is strong enough for the vast majority of real-world use, though VMs do offer a stronger isolation boundary for the highest-security cases.

9. "What If?" Variations

- What if you need to run a Windows-based tool but you're on a Mac? Containers share the host kernel, so this is a case where a VM (or Docker's built-in VM layer on Mac, explained next) is genuinely necessary.
- What if two containers on the same machine need completely different Linux kernel versions? This isn't possible — they share the host kernel. You'd need separate VMs for that.

10. Check Your Understanding

- 1 What is the one big architectural difference between a container and a VM?
- 2 Why do containers start faster than VMs?
- 3 Name one situation where a VM is still the better choice.
- 4 In your own words, what does “shares the host kernel” mean?

🔪 PRACTICE

Mini Challenge: Draw (on paper or in a notes app) two boxes: one showing a VM's layers (hardware → host OS → hypervisor → guest OS → app) and one showing a container's layers (hardware → host OS → container runtime → app). This forces the architectural difference to stick.

12. Lesson Summary

- VMs virtualize an entire computer, including a full guest OS.
- Containers share the host machine's OS kernel and only package the application layer.
- This makes containers much faster to start and much lighter on resources.
- VMs still win when you need a fundamentally different OS/kernel or the strongest possible isolation.

💎 THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Explain the room-in-a-house vs. house-in-a-house analogy in your own words.
- Why is a container faster to start than a VM?
- When would you reach for a VM instead of a container?

Lesson 1.3 — Images vs. Containers

Difficulty: BEGINNER

1. What You Will Learn

- Distinguish a Docker image from a Docker container precisely.
- Explain the relationship between the two using an analogy.
- Predict what happens when you run the same image multiple times.

2. Why This Matters

Beginners very commonly use “image” and “container” interchangeably, which causes real confusion the moment you try to debug something. This distinction is foundational to every lesson after this one.

3. The Simple Explanation

An **image** is a blueprint — a read-only template that contains your application code, a runtime (like Python), libraries, and configuration. It doesn't run by itself; it just exists as a file (or set of files) on disk.

A **container** is what you get when you actually *run* that image. It's a live, working instance — the blueprint brought to life. You can start, stop, and delete containers without touching the image they came from, and you can start many containers from the same single image.

4. Key Terms

Term	Simple Definition	Example
Image	A read-only blueprint/template for creating containers.	A python:3.12-slim image with your app code copied in.
Container	A running (or stopped) instance created from an image.	Container id `a1b2c3` — currently running your FastAPI app on port 8000.
Registry	A place where images are stored and shared.	Docker Hub is the default public registry.
Tag	A label identifying a specific version of an image.	python:3.12-slim — “3.12-slim” is the tag.

5. How It Works

- 1 You write a Dockerfile describing how to build your application's environment (Lesson 2.1).
- 2 Docker reads the Dockerfile and produces an **image** — a static, reusable file.
- 3 You run **docker run** on that image, and Docker creates a **container** from it.
- 4 The container is a live process with its own filesystem view, network, and resources.
- 5 You can run the same image again to get a second, completely independent container.

6. Real-World Example

Docker Hub (hub.docker.com) hosts millions of public images — official images for Python, PostgreSQL, Redis, Nginx, and more. When you write `FROM python:3.12-slim` in a Dockerfile, you are starting from an existing image someone else built and maintains, rather than building the Python runtime from scratch.

7. Practical Example

The cookie-cutter analogy makes this concrete: the image is the cookie cutter (one, reusable), and each container is a cookie you cut with it (many, independent, and you can eat/discard one without affecting the cutter). Commands you'll use starting in Level 2:

```
docker images    # lists your blueprints (images)
docker ps -a     # lists your cookies, running or not (containers)
```

8. Common Mistakes

■ COMMON MISTAKE

Deleting a container and expecting the image to be gone too — they're independent.

■ COMMON MISTAKE

Editing files inside a running container and expecting those changes to persist in the image. They won't, unless you explicitly rebuild the image or use a volume (Lesson 2.3).

■ COMMON MISTAKE

Assuming you can only run one container per image. You can run as many as your machine's resources allow.

9. "What If?" Variations

- What if you delete an image while a container built from it is still running? Docker will refuse, or the container keeps running from what's already loaded — you'll need to stop/remove the container first.
- What if you run the same image twice at the same time? You get two fully independent containers, each with their own memory and (by default) their own isolated filesystem.

10. Check Your Understanding

- 1 What is the difference between an image and a container, in one sentence each?
- 2 Can you run more than one container from a single image? Why or why not?
- 3 If you delete a container, does its image disappear too?
- 4 What is a registry, and name one example.

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Without looking back, write the cookie-cutter analogy from memory, in your own words, including which part is reusable and which part is disposable.

12. Lesson Summary

- An image is a static, read-only blueprint.
- A container is a running (or stopped) instance created from an image.
- Many independent containers can be created from one image.
- Images live in registries like Docker Hub and are identified by name + tag.

♦ THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Explain “image vs. container” using an analogy of your own choosing.
- Why can one image produce many containers?
- What is a tag, and why does it matter?

Level 2 — Core Concepts

Lesson 2.1 — The Dockerfile

Difficulty: BEGINNER

1. What You Will Learn

- Explain what a Dockerfile is and what it produces.
- Read and write the core Dockerfile instructions: FROM, WORKDIR, COPY, RUN, EXPOSE, CMD.
- Write a basic Dockerfile for a Python/FastAPI application.

2. Why This Matters

The Dockerfile is the recipe that turns your source code into a portable image. Every Docker workflow starts here — it is the single most important file in this entire course.

3. The Simple Explanation

A Dockerfile is a plain text file containing step-by-step instructions for building an image — like a recipe card. Each line is an instruction: “start from this base,” “copy these files in,” “run this installation command,” “when the container starts, run this.” Docker reads it top to bottom and builds the image one layer at a time.

4. Key Terms

Term	Simple Definition	Example
Dockerfile	A text file of instructions used to build a Docker image.	A file literally named `Dockerfile` in your project root.
FROM	The instruction that sets the base image to build on top of.	FROM python:3.12-slim
WORKDIR	Sets the working directory inside the image for subsequent instructions.	WORKDIR /app
COPY	Copies files from your machine into the image.	COPY requirements.txt .
RUN	Executes a command while building the image (e.g. installing packages).	RUN pip install -r requirements.txt
CMD	The default command that runs when a container starts from this image.	CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]

Term	Simple Definition	Example
Layer	Each instruction in a Dockerfile creates a cached layer, stacked to form the image.	Changing only your app code re-uses the cached `pip install` layer if requirements.txt didn't change.

5. How It Works

- 1 FROM picks a starting point — usually an official base image with a language runtime already installed.
- 2 WORKDIR sets the folder inside the image where following commands will operate.
- 3 COPY brings your project files (code, requirements.txt) from your machine into the image.
- 4 RUN executes setup commands at build time, most commonly installing dependencies.
- 5 EXPOSE documents which port the app will listen on (informational, doesn't publish it — that happens at `docker run`).
- 6 CMD defines what happens when someone finally runs a container from this image.
- 7 Docker builds these instructions into layers, caching each one so rebuilds are fast when only later lines change.

6. Real-World Example

Almost every backend repository on GitHub that's meant to be deployed includes a Dockerfile at its root. It's the standard, language-agnostic way to say “here's exactly how to build and run this service,” understood by every cloud platform, CI/CD pipeline, and teammate.

7. Practical Example

Here is a real, minimal Dockerfile for a FastAPI backend — the kind of stack you've been building:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Notice requirements.txt is copied and installed *before* the rest of the code. This is deliberate: Docker caches each layer, so if you only change your app code (not your dependencies), Docker skips re-running `pip install` on the next build, making builds much faster.

8. Common Mistakes

■ COMMON MISTAKE

Copying all files (`COPY . .`) before installing dependencies — this breaks Docker's layer cache, so every code change forces a full dependency reinstall.

■ COMMON MISTAKE

Forgetting `--host 0.0.0.0` when running a web server in a container — by default many dev servers only listen on localhost, which is unreachable from outside the container.

■ COMMON MISTAKE

Using `CMD` for something that should run once at build time, instead of `RUN`. `RUN` executes during build; `CMD` executes when the container starts.

9. "What If?" Variations

- What if you add a new dependency to `requirements.txt`? The `RUN pip install` layer's cache is invalidated, and it reinstalls — correctly, because something actually changed.
- What if your app needs a system-level tool (like `ffmpeg`), not just a Python package? You'd add a `RUN` instruction using the OS package manager, e.g. `RUN apt-get update && apt-get install -y ffmpeg`, before your Python-specific steps.
- What if you use `CMD` twice in one Dockerfile? Only the last `CMD` takes effect — it doesn't stack like `RUN` does.

10. Check Your Understanding

- 1 What does `FROM` do, and why is it usually the first line?
- 2 Why does copying `requirements.txt` separately from the rest of the code matter?
- 3 What's the difference between `RUN` and `CMD`?
- 4 What does `EXPOSE` actually do — and not do?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Without looking back, write (from memory) a minimal Dockerfile for a Python script called `worker.py` that has no external dependencies. You'll only need `FROM`, `WORKDIR`, `COPY`, and `CMD`.

12. Lesson Summary

- A Dockerfile is a build recipe: a sequence of instructions that produces an image.
- `FROM`, `WORKDIR`, `COPY`, `RUN`, `EXPOSE`, and `CMD` are the core instructions to know first.
- Order matters for build-cache efficiency — install dependencies before copying the rest of your code.
- `CMD` sets the default startup command; `RUN` executes only during the build.

❖ **THINK ABOUT THIS**

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- What is a Dockerfile, in one sentence?
- Why do we copy requirements.txt before the rest of the app code?
- What's the practical difference between RUN and CMD?

Lesson 2.2 — Docker CLI Essentials

Difficulty: BEGINNER

1. What You Will Learn

- Build an image from a Dockerfile using the CLI.
- Run, list, stop, and remove containers.
- Read container logs and open a shell inside a running container.

2. Why This Matters

The Dockerfile describes your app; the CLI is how you actually bring it to life, inspect it, and clean up after it. These commands are used constantly, every single day, once you work with Docker.

3. The Simple Explanation

Think of the Docker CLI as the remote control for everything you learned in Lessons 2.1 and 1.3: ``docker build`` turns a Dockerfile into an image, ``docker run`` turns an image into a container, and a handful of other commands let you inspect, pause, and clean up containers and images.

4. Key Terms

Term	Simple Definition	Example
docker build	Builds an image from a Dockerfile.	<code>docker build -t my-app .</code>
docker run	Creates and starts a container from an image.	<code>docker run -p 8000:8000 my-app</code>
docker ps	Lists running containers (add <code>-a</code> to include stopped ones).	<code>docker ps -a</code>
docker stop / docker rm	Stops a running container / removes a container.	<code>docker stop a1b2c3 && docker rm a1b2c3</code>
docker logs	Shows the output (stdout/stderr) a container has produced.	<code>docker logs -f a1b2c3</code>
docker exec	Runs a command inside an already-running container.	<code>docker exec -it a1b2c3 bash</code>
-p (port mapping)	Connects a port on your machine to a port inside the container.	<code>-p 8000:8000</code> means host port 8000 → container port 8000.

5. How It Works

- 1 Build: ``docker build -t my-app .`` reads the Dockerfile in the current folder (the trailing dot) and tags the resulting image ``my-app``.
- 2 Run: ``docker run -p 8000:8000 my-app`` starts a container from that image and maps port 8000.

- 3 Inspect: ``docker ps`` shows what's currently running; ``docker ps -a`` also shows stopped containers.
- 4 Debug: ``docker logs`` shows what the app has printed — your first stop when something looks wrong.
- 5 Enter: ``docker exec -it bash`` opens an interactive shell inside a running container, useful for poking around.
- 6 Clean up: ``docker stop`` then ``docker rm`` stop and delete a container; ``docker rmi`` deletes an image.

6. Real-World Example

In real teams, these commands (or their Docker Compose equivalents, covered in Lesson 3.2) run dozens of times a day during local development: rebuild, run, check logs, fix a bug, rebuild again. CI/CD pipelines run the same ``docker build`` command automatically on every code push.

7. Practical Example

A full local cycle for a FastAPI app, from a folder containing your Dockerfile:

```
docker build -t taskflow-api .
docker run -d -p 8000:8000 --name taskflow taskflow-api
docker logs -f taskflow
# ... visit http://localhost:8000 in your browser ...
docker stop taskflow
docker rm taskflow
```

The ``-d`` flag runs the container in the background (“detached”) instead of taking over your terminal. ``--name`` gives it a memorable name instead of a random one.

8. Common Mistakes

■ COMMON MISTAKE

Forgetting ``-p`` and wondering why the app is “unreachable” — the container is running, but no port is connected to your machine.

■ COMMON MISTAKE

Running ``docker run`` repeatedly without ``docker rm``, leaving dozens of stopped containers cluttering ``docker ps -a``.

■ COMMON MISTAKE

Confusing ``docker stop`` (pauses it, keeps it) with ``docker rm`` (deletes it entirely). You often need both.

9. "What If?" Variations

- What if two containers both try to use host port 8000? The second ``docker run`` fails with a port-already-in-use error — map it to a different host port instead, e.g. `-p 8001:8000`.
- What if you forget ``-d``? The container runs in your terminal's foreground; closing that terminal window stops it.

- What if you need to change one line of code? Stop/remove the old container, rebuild the image, and run a new container — code inside an image doesn't update itself.

10. Check Your Understanding

- 1 What does the trailing ``.`` mean in ``docker build -t my-app .``?
- 2 What's the difference between ``docker stop`` and ``docker rm``?
- 3 Why is ``docker logs`` usually your first debugging step?
- 4 What does ``-p 8000:8000`` actually connect?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: From memory, write the exact three commands needed to: (1) build an image tagged ``hello``, (2) run it in the background mapping port 5000 on both sides, (3) view its live logs.

12. Lesson Summary

- `docker build` turns a Dockerfile into an image.
- `docker run` creates and starts a container from an image.
- `docker ps/logs/exec` are your core inspection and debugging tools.
- `docker stop` and `docker rm` are separate steps — stopping doesn't delete.

💠 THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Walk through the full cycle from Dockerfile to a running, reachable container, command by command.
- Why would you use `docker exec` instead of `docker logs`?
- What happens if you skip `-p` when running a web server container?

Lesson 2.3 — Volumes and Persistent Data

Difficulty: BEGINNER

1. What You Will Learn

- Explain why data inside a container disappears by default.
- Distinguish a named volume from a bind mount.
- Use `-v` to persist data or live-reload code during development.

2. Why This Matters

A default container's filesystem is thrown away when the container is removed. For anything that needs to survive — a database's data, uploaded files, or your source code during local development — you need volumes.

3. The Simple Explanation

By default, a container's filesystem is like a whiteboard: whatever you write on it disappears the moment the container (whiteboard) is erased/removed. A **volume** is like attaching a permanent notebook to that whiteboard stand — data written to the notebook survives even after the whiteboard itself is wiped and replaced.

4. Key Terms

Term	Simple Definition	Example
Volume	Docker-managed persistent storage that exists independently of any single container's lifecycle.	A named volume storing a PostgreSQL database's files.
Bind Mount	A direct link between a folder on your host machine and a folder inside the container.	Mounting your local <code>./app</code> source folder into the container for live code reload.
Ephemeral	Temporary; disappears when the container is removed.	A container's default writable layer is ephemeral.
-v flag	The docker run flag used to attach a volume or bind mount.	<code>-v pgdata:/var/lib/postgresql/data</code>

5. How It Works

- 1 Without a volume, every container gets its own throwaway writable layer on top of its image.
- 2 When you delete the container, that writable layer — and anything written to it — is gone.
- 3 A named volume (e.g. ``docker volume create pgdata``) is storage Docker manages outside any single container.
- 4 You attach it with ``-v pgdata:/var/lib/postgresql/data``, meaning “connect the volume named `pgdata` to this path inside the container.”

- 5 A bind mount instead points directly at a folder on your host, e.g. ``-v $(pwd)/app:/app``, so edits on your machine appear instantly inside the container.

6. Real-World Example

Running a database in a container without a volume is a classic, painful beginner mistake — restart the container, and every row of data is gone. In real projects, database containers always mount a named volume. Bind mounts, meanwhile, are what let developers edit code on their host machine and see a running container reflect the change immediately, without rebuilding the image.

7. Practical Example

Persisting a Postgres database's data across restarts:

```
docker volume create pgdata
docker run -d --name db -e POSTGRES_PASSWORD=secret \
-v pgdata:/var/lib/postgresql/data -p 5432:5432 postgres:16
```

Live-reloading a FastAPI app's code during development with a bind mount:

```
docker run -d -p 8000:8000 -v $(pwd):/app taskflow-api
```

8. Common Mistakes

■ COMMON MISTAKE

Running a database container with no volume, then being surprised data vanished after a restart.

■ COMMON MISTAKE

Confusing named volumes (Docker-managed, portable) with bind mounts (tied to a specific host path) — they solve different problems.

■ COMMON MISTAKE

Bind-mounting your whole project including a heavy local dependency folder, dramatically slowing container startup.

9. "What If?" Variations

- What if you delete the container but not the volume? The data is untouched — attach a new container to the same volume and it's all still there.
- What if two containers mount the same named volume at the same time? Both can read/write it, but you're responsible for making sure that's safe for your specific app (databases generally should not be run this way).
- What if you bind-mount a folder that doesn't exist on your host yet? Docker will create it as an empty folder automatically.

10. Check Your Understanding

- 1 Why does data disappear by default when a container is removed?
- 2 What's the practical difference between a named volume and a bind mount?
- 3 Why would you never run a production database container without a volume?
- 4 When would a bind mount be more useful than a named volume?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Explain, without notes, what would happen to your data in three scenarios: (1) no volume, container removed; (2) named volume, container removed; (3) named volume, volume itself deleted.

12. Lesson Summary

- Containers are ephemeral by default — their writable layer disappears with them.
- Named volumes give you Docker-managed, persistent storage independent of any one container.
- Bind mounts link a host folder directly into a container — great for live development.
- Databases in containers should always use a volume.

♦ THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Why is a database container without a volume dangerous?
- Explain the whiteboard/notebook analogy in your own words.
- When would you choose a bind mount over a named volume?

Review Session 1

Covers: Lessons 1.1 – 2.3 (Foundations and Core Concepts)

Easy Questions

- 1 What does Docker package together?
- 2 What command builds an image from a Dockerfile?
- 3 What happens to data in a container's default writable layer when the container is removed?

Medium Questions

- 1 Explain the difference between an image and a container, and between a container and a VM, in one paragraph total.
- 2 Why should requirements.txt be copied and installed before the rest of your app code in a Dockerfile?
- 3 What's the difference between docker stop and docker rm, and why would you need both?

Hard Questions

- 1 A teammate says: "I added a volume to my database container, but after `docker rm` and a fresh `docker run`, the data is still gone." List two possible mistakes that would cause this.
- 2 Explain why containers start in roughly a second while VMs take tens of seconds, connecting it back to what each one virtualizes.
- 3 You changed one line of Python code but not requirements.txt, and a full rebuild still reinstalled every dependency from scratch. What's the most likely Dockerfile mistake, and how would you fix it?

♦ THINK ABOUT THIS

What Should I Still Remember?

- Docker solves "it works on my machine" by packaging the app with its exact environment.
- Containers share the host OS kernel; VMs virtualize an entire OS.
- Images are static blueprints; containers are running instances.
- FROM, WORKDIR, COPY, RUN, EXPOSE, CMD — the six core Dockerfile instructions.
- Volumes persist data beyond a container's lifecycle; bind mounts link a host folder directly in.

Knowledge Checkpoint 1: Foundations and Core Concepts

Answer honestly, then use the scale below to decide whether to move forward.

- 1 Can you explain, without notes, why Docker exists?
- 2 Can you explain the difference between an image and a container without hesitating?
- 3 Can you write a basic Dockerfile for a simple Python app from memory?
- 4 Can you list the commands to build, run, and stop a container without looking them up?
- 5 Do you understand why a database container needs a volume?

■ Ready	You answered all five confidently and could explain them to someone else. Move to Level 3.
■ Review Needed	You're shaky on 1–2 of the five. Re-read that specific lesson's “Simple Explanation” and “Can You Explain It?” sections before continuing.
■ Not Ready	Three or more felt unclear. Go back through Lessons 1.1–2.3 in full, redoing the Mini Challenges, before moving on.

Level 3 — Practical Usage

Level 4 (“Problem Solving”) from the standard course template is woven directly into this level's exercises and into Review Session 2, rather than given a separate set of lessons — for Docker, problem-solving is best learned attached to real commands, not in isolation.

Lesson 3.1 — Building Your First Real Image

Difficulty: BEGINNER

1. What You Will Learn

- Take an existing FastAPI project and containerize it end to end.
- Diagnose the three most common first-build failures.
- Confirm a containerized app is reachable from a browser.

2. Why This Matters

Reading about Dockerfiles and commands is not the same as doing it. This lesson is the first moment where the pieces from Level 2 combine into something you can actually run and see work.

3. The Simple Explanation

You already know each individual piece: a Dockerfile describes the environment (2.1), the CLI builds and runs it (2.2), and volumes can keep code live during development (2.3). This lesson is about assembling those pieces, in order, against a real project, and — just as importantly — knowing what to do when it doesn't work the first time (it usually doesn't).

4. Key Terms

Term	Simple Definition	Example
Build context	The set of files Docker can see and copy from during a build — normally the folder you run <code>`docker build`</code> in.	Running <code>`docker build .`</code> from your project root makes that whole folder the build context.
<code>`.dockerignore`</code>	A file listing paths to exclude from the build context, similar to <code>`.gitignore`</code> .	Excluding <code>`venv/`</code> , <code>`__pycache__`</code> , and <code>`.git`</code> so they aren't copied into the image.
Port collision	When two processes try to use the same network port at once.	Container A and Container B both trying to publish to host port 8000.

5. How It Works

- 1 Start from a working, already-tested local project (don't containerize and debug app logic at the same time).
- 2 Add a `requirements.txt` if you don't already have one (e.g. `pip freeze > requirements.txt`).
- 3 Write the Dockerfile as in Lesson 2.1.
- 4 Add a `.dockerignore` excluding virtual environments, caches, and version control folders.
- 5 Run `docker build -t .`` and read any errors carefully — they usually point at the exact failing line.
- 6 Run the container with `-p`` mapping a port, then open that port in your browser or with `curl``.
- 7 If it fails, check `docker logs`` before anything else.

6. Real-World Example

This exact sequence — write Dockerfile, build, run, hit an error, check logs, fix, rebuild — is what containerizing any real backend service looks like on day one, at any company. Nobody gets it right on the first try; the skill is debugging it quickly.

7. Practical Example

A `.dockerignore`` for a typical Python backend:

```
__pycache__/  
*.pyc  
.venv/  
venv/  
.git/  
.env  
.pytest_cache/  
*.db
```

The full first-build cycle:

```
docker build -t taskflow-api .  
docker run -d -p 8000:8000 --name taskflow taskflow-api  
curl http://localhost:8000/health  
docker logs taskflow    # if curl fails or hangs
```

8. Common Mistakes

■ COMMON MISTAKE

No `.dockerignore``, so a huge local virtual environment folder gets copied into the build context, making builds extremely slow.

■ COMMON MISTAKE

The app binds to `127.0.0.1`` instead of `0.0.0.0`` inside the container — it works from `docker exec`` inside the container but is unreachable from the host.

■ COMMON MISTAKE

Forgetting that `.env` files with local secrets often shouldn't be baked into an image at all — covered fully in Lesson 5.2.`

9. "What If?" Variations

- What if ``docker build` fails on the `pip install` line? Read the actual error — it's almost always a missing system dependency (fixable with an `apt-get install` RUN line) or a version conflict in requirements.txt.`
- What if the build succeeds but ``curl` hangs? Check the app is listening on 0.0.0.0, and check the port after `-p` matches what the app actually listens on inside the container.`
- What if the container exits immediately after ``docker run`? Run `docker logs` on it — it almost always shows a Python traceback explaining exactly what crashed.`

10. Check Your Understanding

- 1 What is a build context, and why does a `.dockerignore` matter for it?`
- 2 Your container starts and then immediately stops. What's your first command to investigate?
- 3 You can ``curl` the app from inside the container with `docker exec` but not from your host browser. What's the likely cause?`
- 4 Why should app logic already be tested locally before you try to containerize it?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Pick any small script or app you've already written (it doesn't need to be complex). Write a Dockerfile for it, build it, run it, and confirm it works — before reading Lesson 3.2.

12. Lesson Summary

- Containerizing a real app is assembling Dockerfile + CLI + (optionally) volumes against tested code.
- A `.dockerignore` keeps builds fast and images small.`
- ``docker logs` is your first, best debugging tool for a failing container.`
- 0.0.0.0 vs 127.0.0.1 inside a container is one of the most common “works internally, unreachable externally” bugs.

♦ THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Walk through your full first-build cycle, from an untouched project folder to a working container, step by step.
- What are the first three things you check when a container won't start?
- Why does binding to 0.0.0.0 matter inside a container specifically?

Lesson 3.2 — Docker Compose Basics

Difficulty: BEGINNER

1. What You Will Learn

- Explain what problem Docker Compose solves that plain ``docker run`` doesn't.
- Read and write a basic `docker-compose.yml` with two services.
- Start, stop, and rebuild a multi-container app with Compose.

2. Why This Matters

Real applications are rarely a single container — a typical backend needs the app itself plus at least a database. Running each with separate, long ``docker run`` commands is tedious and error-prone. Compose lets you describe your whole application stack in one file.

3. The Simple Explanation

If a single Dockerfile + ``docker run`` is a recipe for one dish, Docker Compose is the full menu for a meal with several dishes that need to be served together — the app, the database, maybe a cache — all started, connected, and stopped as one unit with a single command.

4. Key Terms

Term	Simple Definition	Example
Docker Compose	A tool for defining and running multi-container applications from a single YAML file.	<code>docker-compose.yml</code> defining both a <code>`web`</code> service and a <code>`db`</code> service.
Service	One container definition within a Compose file.	The <code>`web`</code> service builds from your Dockerfile; the <code>`db`</code> service uses the official postgres image.
docker-compose.yml	The YAML file where services, networks, and volumes are declared.	Placed at your project root, alongside your Dockerfile.
<code>`docker compose up`</code>	Builds (if needed) and starts every service defined in the file.	<code>docker compose up -d</code>

5. How It Works

- 1 Create a ``docker-compose.yml`` at your project root.
- 2 Define each piece of your app as a ``service`` — your API, a database, etc.
- 3 Each service specifies either ``build: .`` (build from your Dockerfile) or ``image: `` (use an existing image, like ``postgres:16``).
- 4 Compose automatically creates a shared network so services can reach each other by service name (fully explained in Lesson 5.1).

- 5 Run ``docker compose up`` to start everything together; ``docker compose down`` to stop and remove it all.

6. Real-World Example

A typical FastAPI + PostgreSQL local development setup, like the kind used for your TaskFlow-style projects, is almost always run through Compose rather than two separate long ``docker run`` commands — it's the difference between one command and remembering five flags across two terminals every time you start work.

7. Practical Example

A `docker-compose.yml` for a FastAPI app plus a PostgreSQL database:

```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
    environment:
      DATABASE_URL: postgresql://user:pass@db:5432/taskflow

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: taskflow
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Bring the whole stack up, view logs, and tear it down:

```
docker compose up -d
docker compose logs -f web
docker compose down
```

8. Common Mistakes

■ COMMON MISTAKE

Using ``localhost`` for ``DATABASE_URL`` instead of the service name ``db`` — inside Compose's network, containers reach each other by service name, not localhost (Lesson 5.1 explains why).

■ COMMON MISTAKE

Forgetting ``depends_on``, causing the web service to try connecting to a database that hasn't started yet.

■ COMMON MISTAKE

Running ``docker compose down -v`` out of habit and accidentally deleting the database volume along with the containers — the ``-v`` flag also removes volumes.

9. "What If?" Variations

- What if you only want to restart the web service, not the whole stack? ``docker compose restart web`` targets just that one service.
- What if you change your Dockerfile? ``docker compose up`` won't rebuild automatically — use ``docker compose up --build`` (or ``docker compose build`` first) to force a rebuild.
- What if ``depends_on`` starts the db container first but the database inside it isn't ready to accept connections yet? ``depends_on`` only waits for the container to start, not for the app inside it to be ready — real projects add a healthcheck or retry logic to handle this, covered in the Final Project.

10. Check Your Understanding

- 1 What problem does Compose solve that individual ``docker run`` commands don't?
- 2 Inside a Compose network, how does the web service refer to the database service?
- 3 What does ``docker compose down -v`` do differently from ``docker compose down``?
- 4 Why doesn't ``depends_on`` alone guarantee the database is truly ready?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Extend the `docker-compose.yml` above by adding a third service called ``redis`` using the official ``redis:alpine`` image, exposing port 6379. Don't run it — just write the YAML from memory first, then check it against the pattern used for ``db``.

12. Lesson Summary

- Docker Compose defines a multi-container app in one YAML file.
- Each container is a ``service``; services share a network and can reach each other by name.
- ``docker compose up`` / ``down`` start and stop the entire stack together.
- ``depends_on`` controls start order, not full application readiness.

♦ THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Why is Compose better than several manual `docker run` commands for a real app?
- How does the web service find the database service by name?
- What's the risk of running ``docker compose down -v`` carelessly?

Level 5 — Intermediate Concepts

Lesson 5.1 — Networking Between Containers

Difficulty: DEVELOPING

1. What You Will Learn

- Explain how containers on the same Compose network find each other.
- Distinguish the host port mapping from the internal container port.
- Debug a “connection refused” error between two services.

2. Why This Matters

Almost every real app is more than one container, and those containers need to talk to each other — an API talking to a database is the most common example. Misunderstanding container networking is one of the single biggest sources of beginner confusion and bug reports.

3. The Simple Explanation

Imagine an office building where every company (container) has its own room, but the building directory lets any company call any other company's room just by name, without knowing the exact phone number. Docker Compose builds exactly this kind of private directory: every service can reach every other service by its *service name*, over an internal network invisible to the outside world unless you explicitly open a door (`ports:`) to it.

4. Key Terms

Term	Simple Definition	Example
Bridge network	The default type of private network Docker creates for containers to talk to each other.	Docker Compose automatically creates one bridge network per project.
Service discovery	The ability for containers to find each other by name instead of IP address.	The web service connects to <code>`db:5432`</code> , not to a raw IP address.
Published port	A port explicitly exposed to the host machine via <code>`ports:`</code> or <code>`-p`</code> .	<code>`ports: ["8000:8000"]`</code> makes the API reachable from your browser.
Internal-only port	A port reachable by other containers on the same network, but not from the host, unless separately published.	The db service's 5432 port doesn't need to be published for <code>`web`</code> to reach it.

5. How It Works

- 1 When Compose starts, it creates a private network shared by all services in that file.
- 2 Each service is automatically reachable by other services on that network using its service name as a hostname.
- 3 A connection string like ``postgresql://user:pass@db:5432/taskflow`` works because ``db`` resolves to the database container's internal IP.
- 4 ``ports: ["8000:8000"]`` (format is ``host:container``) is what makes a service reachable from outside Docker entirely — from your browser or ``curl`` on your own machine.
- 5 A service that other containers need but your host machine does not (like a database, in many setups) doesn't strictly need a ``ports:`` entry at all.

6. Real-World Example

This is exactly how a FastAPI backend reaches a PostgreSQL database in virtually every containerized project: the app's ``DATABASE_URL`` environment variable points at the database's *service name*, never at ``localhost`` and never at a hardcoded IP address, because container IPs can change on restart.

7. Practical Example

Given the Compose file from Lesson 3.2, this connection string works from inside the ``web`` container:

```
DATABASE_URL=postgresql://user:pass@db:5432/taskflow
```

But this would NOT work from inside a container (only from your host machine's browser, and only because port 8000 was published):

```
http://localhost:8000
```

You can prove two services can see each other with a quick check:

```
docker compose exec web ping db
```

8. Common Mistakes

■ COMMON MISTAKE

Using ``localhost`` inside one container to reach another container — ``localhost`` inside a container refers to that container itself, not the host machine or any other container.

■ COMMON MISTAKE

Publishing every service's port to the host “just in case,” increasing your attack surface for no reason — only publish ports you actually need to reach from outside Docker.

■ COMMON MISTAKE

Hardcoding a container's IP address instead of using its service name — container IPs are not guaranteed to stay the same across restarts.

9. "What If?" Variations

- What if two Compose projects both try to use a service named `db`? They're isolated in separate networks by default (one per project), so there's no conflict.
- What if you need a container to reach a service running directly on your host machine (not in Docker)? On Docker Desktop, the special hostname `host.docker.internal` resolves to your host machine.
- What if `web` starts before `db` is actually ready to accept connections, even with `depends_on`? You'll see connection-refused errors on startup — this is exactly the readiness gap mentioned in Lesson 3.2, solved properly with healthchecks in the Final Project.

10. Check Your Understanding

- 1 Why does `db:5432` work as a hostname from inside another container on the same Compose network?
- 2 What's the difference between a port being published and a port only being reachable internally?
- 3 Why is using `localhost` to reach another container a mistake?
- 4 What would `docker compose exec web ping db` prove if it succeeds?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Without notes, explain why an API container can reach `db:5432` but your laptop's browser cannot, unless db's port is separately published.

12. Lesson Summary

- Compose gives every service in a project a shared private network and automatic name-based discovery.
- Services reach each other by service name, never by localhost or hardcoded IP.
- `ports:` is specifically what exposes a service to the host machine, not what connects services to each other.
- Container startup order (`depends_on`) is not the same as application readiness.

💎 THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Explain how one container finds another by name, using the office-building analogy or your own.
- Why would a database service often skip a `ports:` entry entirely in production-style setups?
- What's the difference between localhost inside a container and localhost on your host machine?

Lesson 5.2 — Environment Variables and Secrets

Difficulty: *DEVELOPING*

1. What You Will Learn

- Pass configuration into a container without hardcoding it in the image.
- Explain why secrets shouldn't be baked into a Dockerfile.
- Use a `.env` file with Docker Compose correctly.

2. Why This Matters

Hardcoding a database password or API key directly into your Dockerfile or source code means it's baked into the image itself — visible to anyone who can pull or inspect that image, and a nightmare to rotate. Environment variables solve this by injecting configuration at run time instead.

3. The Simple Explanation

Think of your image as a printed, unchangeable form letter, and environment variables as sticky notes you attach right before mailing it — the letter itself never has to be reprinted (rebuilt) just because a value like an API key or a target environment changes.

4. Key Terms

Term	Simple Definition	Example
Environment variable	A named value passed into a running process from outside it, rather than hardcoded.	DATABASE_URL, SECRET_KEY, DEBUG=false
.env file	A plain text file of KEY=value pairs, commonly read automatically by Docker Compose.	SECRET_KEY=supersecret DEBUG=false
Secret	Sensitive configuration — passwords, API keys, tokens — that must never be committed to version control or baked into an image.	A JWT signing key or a third-party API key.
<code>.gitignore</code> / <code>.dockerignore</code>	Files that prevent sensitive or unnecessary files from being committed or copied into a build.	Adding <code>.env</code> to both, so secrets never reach GitHub or an image layer.

5. How It Works

- 1 Never write a real secret directly into a Dockerfile with RUN or ENV — it becomes a permanent part of every image layer, even if later “overwritten.”
- 2 Define configuration in a `.env` file at your project root, and add `.env` to both `.gitignore` and `.dockerignore`.
- 3 Docker Compose automatically reads a `.env` file in the same folder as your `docker-compose.yml`.

- 4 Reference those variables inside ``docker-compose.yml`` using ``${VARIABLE_NAME}`` syntax, or list them directly under a service's ``environment:`` key.
- 5 Inside your application code, read them the normal way for your language (e.g. Python's ``os.environ.get("SECRET_KEY")``).

6. Real-World Example

This is exactly how real teams manage differences between local development, staging, and production — the same image is deployed everywhere, and only the environment variables change per environment. Cloud platforms like Render, Railway, and AWS all have a dedicated “environment variables” or “secrets” panel built around this exact pattern.

7. Practical Example

A ``.env`` file at your project root:

```
POSTGRES_USER=user
POSTGRES_PASSWORD=pass
POSTGRES_DB=taskflow
SECRET_KEY=change-me-in-real-life
```

Referencing it inside `docker-compose.yml`:

```
services:
  web:
    build: .
    environment:
      DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
      SECRET_KEY: ${SECRET_KEY}
  db:
    image: postgres:16
    env_file:
      - .env
```

8. Common Mistakes

■ COMMON MISTAKE

Writing ``ENV SECRET_KEY=abc123`` directly in a Dockerfile — that value is now permanently baked into the image, retrievable by anyone with access to it.

■ COMMON MISTAKE

Committing a ``.env`` file with real secrets to a public GitHub repository — one of the most common real-world security incidents.

■ COMMON MISTAKE

Assuming a value is gone from an image just because a later Dockerfile layer overwrites it — earlier layers (and their secrets) can often still be inspected.

9. "What If?" Variations

- What if different teammates need different local values (e.g. different API keys)? Each person keeps their own local `.env`` (never committed), while a `.env.example`` with placeholder keys IS committed, documenting what's needed.
- What if you need secrets in an actual production deployment, not just locally? Real production setups typically use a secrets manager (like AWS Secrets Manager or Docker Swarm/Kubernetes secrets) instead of a plain `.env`` file — outside the scope of this course, but good to know it exists.
- What if you forget to add `.env`` to `.gitignore`` before your first commit? Removing it from a future commit does NOT remove it from your Git history — the secret must be rotated (changed) immediately.

10. Check Your Understanding

- 1 Why is `ENV SECRET_KEY=abc123`` in a Dockerfile a security problem?
- 2 What's the difference between `.env`` and `.env.example``?
- 3 How does Docker Compose automatically use a `.env`` file?
- 4 If a secret was accidentally committed to Git and later removed, is it safe? Why or why not?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Take the `docker-compose.yml` from Lesson 3.2 and rewrite it from memory so that the database password comes from a `.env`` file instead of being hardcoded directly in the YAML.

12. Lesson Summary

- Secrets should never be hardcoded into a Dockerfile or committed to version control.
- A `.env`` file, excluded from Git and Docker builds, is the standard local pattern.
- Docker Compose reads `.env`` automatically and supports ``${VARIABLE}`` substitution.
- A committed secret must be rotated, not just deleted from a future commit.

💠 THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Explain why baking a secret into a Dockerfile is dangerous, using the sticky-note analogy or your own.
- What's the purpose of a `.env.example`` file?
- What should you do the moment you realize a real secret was committed to GitHub?

Lesson 5.3 — Multi-Stage Builds

Difficulty: INTERMEDIATE

1. What You Will Learn

- Explain why final image size matters.
- Write a Dockerfile using multiple FROM stages.
- Identify when multi-stage builds are worth the added complexity.

2. Why This Matters

Some build steps — compiling code, installing build tools — are only needed to *produce* your app, not to *run* it. Shipping those build-only tools in your final image wastes space and increases your attack surface for no benefit. Multi-stage builds let you throw that scaffolding away.

3. The Simple Explanation

Think of building a piece of furniture: you need a full workshop — saws, sanders, glue, clamps — to build it, but once it's built, you only ship the finished chair, not your entire workshop. A multi-stage Dockerfile has one stage that's the workshop (with all the build tools) and a final stage that only copies over the finished product.

4. Key Terms

Term	Simple Definition	Example
Stage	One <code>FROM</code> block in a Dockerfile; a multi-stage build has more than one.	A <code>builder</code> stage that compiles, and a final stage that runs.
<code>AS</code>	Names a stage so you can reference it later in the same Dockerfile.	<code>FROM python:3.12 AS builder</code>
<code>COPY --from=</code>	Copies files from a named earlier stage into the current one.	<code>COPY --from=builder /app/dist /app</code>
Image size	How large the final built image is on disk — smaller is generally faster to pull, deploy, and more secure.	A multi-stage Python build might shrink an image from 1.2GB to 200MB.

5. How It Works

- 1 Define a first stage (e.g. `FROM python:3.12 AS builder`) that includes everything needed to build or compile your app.
- 2 Do the heavy lifting there — installing build-time dependencies, compiling, etc.
- 3 Start a second, final stage from a minimal base image (e.g. `python:3.12-slim`).
- 4 Use `COPY --from=builder` to pull only the finished artifacts into the final stage — not the build tools that produced them.

- 5 The final stage becomes your actual shipped image; the builder stage is discarded after the build finishes.

6. Real-World Example

This pattern is standard for compiled languages (Go, Rust, Java) where the compiler itself is enormous but produces a single small binary. It's increasingly common in Python and Node projects too, whenever a build step (like compiling a frontend, or building wheels for packages with C extensions) is involved.

7. Practical Example

A simplified multi-stage Dockerfile for a Python app with a compiled dependency:

```
# Stage 1: build
FROM python:3.12 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# Stage 2: final, minimal runtime
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

8. Common Mistakes

■ COMMON MISTAKE

Using multi-stage builds for a simple app with no heavy build step — it's added complexity that isn't always worth it. Reach for it when it solves a real size or security problem.

■ COMMON MISTAKE

Forgetting `--from=` and accidentally copying from the host machine instead of the named stage.

■ COMMON MISTAKE

Copying more than needed from the builder stage (e.g. the entire `/app` folder including build-only files) and losing most of the size benefit.

9. "What If?" Variations

- What if your app has no compiled dependencies and installs quickly? A single-stage Dockerfile (Lesson 2.1) is simpler and perfectly fine — multi-stage is an optimization, not a requirement.
- What if you need three distinct steps (e.g. compile frontend, compile backend, then assemble)? You can chain more than two stages, naming each and copying selectively between them.

- What if the final, slim base image is missing a system library the app needs at runtime (even though it's not needed at build time)? You'd add a small RUN apt-get install line in the final stage specifically for that runtime dependency.

10. Check Your Understanding

- 1 Why would you want to discard the tools used to build your app from the final image?
- 2 What does `COPY --from=builder`` do?
- 3 When would a single-stage Dockerfile be the better, simpler choice?
- 4 Why might a multi-stage build reduce security risk, not just image size?

Answers are not given here — check the Answer Key at the end of the relevant Review Session or the Course Answer Key.

🔪 PRACTICE

Mini Challenge: Estimate (roughly, no need to actually build it) how much smaller a multi-stage image might be for an app that needs a full compiler toolchain at build time but none of it at runtime. Explain your reasoning.

12. Lesson Summary

- Multi-stage builds separate build-time tools from the final runtime image.
- `COPY --from=` pulls only the finished artifacts forward.
- Smaller final images are faster to deploy and reduce attack surface.
- Not every project needs this — it's a targeted optimization for build-heavy apps.

💠 THINK ABOUT THIS

Can You Explain It? Try explaining this concept out loud or in writing, without looking back at the lesson:

- Explain the workshop/finished-furniture analogy in your own words.
- Walk through what happens to the ``builder`` stage after the final image is built.
- When would you NOT bother with a multi-stage build?

Review Session 2

Covers: Lessons 3.1 – 5.3 (Practical Usage and Intermediate Concepts)

Easy Questions

- 1 What file lists paths to exclude from a Docker build context?
- 2 In Compose, how does one service refer to another by name?
- 3 What command starts every service defined in a docker-compose.yml?

Medium Questions

- 1 Why does `DATABASE_URL` inside a container use the service name `db`, never `localhost`?
- 2 Explain the difference between a port being published with `ports:` and a port only being reachable inside the Compose network.
- 3 Why should a real secret never be written directly into a Dockerfile with ENV?

Hard Questions

- 1 A teammate's `web` service crashes on startup with a database connection error, even though `depends_on: [db]` is set. Explain, in detail, why `depends_on` alone doesn't guarantee this works, and what you'd add to fix it.
- 2 Design (in words, not code) a multi-stage Dockerfile for a project that needs to compile a frontend build step and then serve it from a minimal Python backend image. What goes in each stage?
- 3 A `.env` file with a real production database password was committed to a public repo and later deleted in a new commit. Explain exactly why the team still isn't safe, and what they must do.

◆ THINK ABOUT THIS

What Should I Still Remember?

- A `.dockerignore` and testing app logic before containerizing avoid most first-build failures.
- Compose services reach each other by service name over a shared private network; `ports:` is what reaches the host.
- Secrets belong in `.env` files excluded from Git and Docker builds — never hardcoded in a Dockerfile.
- `depends_on` controls start order, not application readiness — healthchecks close that gap (Final Project).
- Multi-stage builds discard build-only tools from the final shipped image.

Knowledge Checkpoint 2: Practical Usage and Intermediate Concepts

Answer honestly, then use the scale below to decide whether to move forward.

- 1 Can you containerize a simple app end to end without referring back to the lessons?
- 2 Can you explain why `localhost` doesn't work between two containers?
- 3 Can you write a `docker-compose.yml` with two connected services from memory?
- 4 Do you know where secrets belong, and where they must never go?
- 5 Can you explain what a multi-stage build actually discards, and why that matters?

■ Ready	All five felt solid. You're ready for the Mini Projects and the Final Project.
■ Review Needed	One or two felt shaky — revisit that specific lesson's Practical Example and redo its Mini Challenge before starting the projects.
■ Not Ready	Three or more felt unclear. Redo Lessons 3.1–5.3 fully, actually typing every command, before attempting the projects — the projects assume this level is solid.

Level 6 — Mini Projects

Each project below gives you less hand-holding than the one before it — that's intentional. The goal is to gradually train you to solve problems with only a Dockerfile, the CLI, and your own reasoning, not a lesson to copy from.

Project — Dockerize a FastAPI Backend (Guided)

Objective

Take a working FastAPI application (your own, or a simple one you write for this project) and containerize it so it runs identically for anyone with Docker installed.

Requirements

- A FastAPI app with at least one working endpoint, tested and running locally first.
- A requirements.txt listing its dependencies.
- A Dockerfile that builds and runs it.
- A .dockerignore excluding unnecessary files.

Concepts Being Tested

- Lesson 2.1 (Dockerfile)
- Lesson 2.2 (CLI essentials)
- Lesson 3.1 (first real build)

Step-by-Step Guidance

- 1 Confirm the app runs correctly with ``uvicorn main:app --reload`` locally, outside Docker, first.
- 2 Generate requirements.txt if you don't have one.
- 3 Write a Dockerfile following the Lesson 2.1 pattern.
- 4 Write a .dockerignore excluding your virtual environment, caches, and .git.
- 5 Build the image and run a container, mapping a port.
- 6 Confirm the endpoint responds from your host machine's browser or curl — not just from inside the container.

Hints

- If the build is slow, check whether your .dockerignore is actually excluding your venv folder.
- If curl hangs, check the app is bound to 0.0.0.0, not 127.0.0.1.
- Use ``docker logs`` before anything else if the container exits immediately.

Common Mistakes

■ COMMON MISTAKE

Trying to debug application logic and containerization problems at the same time — isolate them.

■ COMMON MISTAKE

Forgetting the `-p`` flag entirely and assuming the app is broken when it's just unreachable.

Testing Checklist

- docker build completes with no errors.
- docker run starts the container and it stays running (check with `docker ps`, not just the initial output).
- The endpoint responds correctly from your host machine.
- docker logs shows no unexpected errors.

Expected Result

A single image that, when run with one ``docker run`` command and a port mapping, serves your API identically to how it ran locally.

Extension Challenges

- Add a ``/health`` endpoint and confirm it responds from the container.
- Reduce the image size by switching to a ``-slim`` base image if you weren't already using one.
- Add a bind mount so code changes reload without rebuilding the image.

Project — FastAPI + PostgreSQL with Docker Compose (Semi-Guided)

Objective

Extend the previous project into a two-service stack: your FastAPI app and a real PostgreSQL database, connected and persisted correctly, defined entirely in Docker Compose.

Requirements

- A docker-compose.yml defining a `web` service and a `db` service.
- A named volume so database data survives container restarts.
- Secrets and configuration passed via a .env file, not hardcoded.
- The web service actually able to query the database successfully.

Concepts Being Tested

- Lesson 3.2 (Compose basics)
- Lesson 5.1 (networking)
- Lesson 5.2 (env vars and secrets)
- Lesson 2.3 (volumes)

Step-by-Step Guidance

- 1 Write docker-compose.yml with two services, following the Lesson 3.2 pattern as a starting reference — don't just copy it verbatim, adapt it.
- 2 Add a named volume for the db service's data directory.
- 3 Move all credentials into a .env file, referenced with \${VARIABLE} syntax.
- 4 Set DATABASE_URL on the web service to point at the db service by name, not localhost.
- 5 Bring the stack up with `docker compose up -d` and confirm both containers are running.
- 6 From your app, perform a real database operation (e.g. create a row) and confirm it works.
- 7 Stop the stack, bring it back up, and confirm the data is still there.

Hints

- If the web service can't reach the database, double-check you're using the service name, not localhost, in the connection string.
- If data disappears after `docker compose down` + `up`, check you didn't accidentally use the `-v` flag on `down`, and confirm the volume is actually declared and mounted.

Common Mistakes

■ COMMON MISTAKE

Using localhost instead of the service name in DATABASE_URL.

■ COMMON MISTAKE

Running `docker compose down -v` out of habit and wiping the volume.

■ COMMON MISTAKE

Hardcoding the database password directly in docker-compose.yml instead of pulling it from .env.

Testing Checklist

- docker compose up -d starts both services without errors.
- The app can successfully read/write to the database.
- Restarting the stack (down, then up) preserves existing data.
- No secret values appear directly in docker-compose.yml — only \${VARIABLE} references.

Expected Result

Running `docker compose up -d` from a clean checkout (with a .env file in place) gives you a fully working API backed by a real, persistent PostgreSQL database.

Extension Challenges

- Add a third service: a lightweight Adminer or pgAdmin container for browsing the database visually.
- Add a `.env.example` file documenting required variables without real values.
- Try deliberately breaking depends_on's assumption — add an artificial delay to the app's startup and observe what happens on a fresh `up`.

Level 7 — Final Challenge

This project gives you the least guidance of the course. You're given requirements, not steps. This is deliberate — it's the closest simulation of a real task you'd be handed at a job.

Project — Production-Style Multi-Service Stack (Independent)

Objective

Design and build a Docker Compose stack for a FastAPI backend and a PostgreSQL database that is closer to production-ready than the previous project: correctly ordered startup, a small final image, and no secrets in version control.

Requirements

- A multi-stage Dockerfile for the web service, producing the smallest reasonable final image.
- A healthcheck on the db service so the web service only starts after the database can actually accept connections — not just after its container has started.
- All secrets sourced from a `.env` file, with a committed `.env.example` documenting what's required.
- A named volume for persistent data.
- A `.dockerignore` and a `.gitignore`, both correctly excluding secrets and build artifacts.

Concepts Being Tested

- Everything from Levels 2, 3, and 5, combined.

Step-by-Step Guidance

- 1 This is intentionally open — you decide the order. Start from the Mini Project 2 stack and improve it piece by piece.
- 2 Research (or recall from Lesson 5.1) how Compose healthchecks work: a ``healthcheck:`` block on the db service, referenced by ``condition: service_healthy`` under the web service's ``depends_on``.
- 3 Convert the web service's Dockerfile into a multi-stage build.
- 4 Verify: stop the whole stack, delete all containers (but not the volume), and bring it back up from scratch — it should self-heal into a fully working state with zero manual steps.

Hints

- A Postgres healthcheck typically runs ``pg_isready`` inside the db container on an interval.
- ``depends_on`` in Compose supports a long form with ``condition: service_healthy``, not just a plain list of service names.
- Measure your image size before and after the multi-stage conversion with ``docker images`` to confirm it actually helped.

Common Mistakes

■ COMMON MISTAKE

Adding a healthcheck but forgetting to reference it with `condition: service_healthy`` in `depends_on` — the healthcheck alone doesn't change startup order by itself.

■ COMMON MISTAKE

Multi-staging the Dockerfile but still copying the entire builder stage's filesystem into the final image, losing the size benefit.

Testing Checklist

- `docker compose up -d` from a completely clean state works with zero manual intervention.
- The web service never throws a database-connection error on a fresh start, even on a slow machine.
- `docker images` shows the final web image meaningfully smaller than a single-stage equivalent.
- `git status` (or a manual check) confirms no secret values are tracked in version control.

Expected Result

A stack that a new teammate could clone, add their own `.env` based on your `.env.example`, and run with one command — with zero manual troubleshooting steps required.

Extension Challenges

- Add a basic GitHub Actions workflow (or just describe one) that builds the image on every push.
- Research what changes would be needed to deploy this stack to a real cloud platform.
- Write a short README explaining the setup for a future teammate — explaining, not just listing, commands.

Final Exam

This tests whether you can use what you learned, not just recall it. Write full answers before checking the Answer Key that follows — don't peek early.

Part 1 — Concept Questions

- 1 What is the core problem Docker solves?
- 2 What is the architectural difference between a container and a virtual machine?
- 3 What is the relationship between a Docker image and a Docker container?
- 4 What does a Dockerfile produce when built?

Part 2 — Explain In Your Own Words

- 1 Explain, as if to a beginner, why a database container needs a volume.
- 2 Explain why two containers on the same Compose network use service names instead of localhost.
- 3 Explain what a multi-stage build removes from the final image, and why that's valuable.

Part 3 — Practical Problems

- 1 Write a minimal Dockerfile for a Python script with no external dependencies, from scratch.
- 2 Write the docker-compose.yml for a FastAPI app and a Redis cache (image: redis:alpine), including a published port for the app only.
- 3 Write the exact command to see the live logs of a running container named `api`.

Part 4 — Scenario Questions

- 1 A teammate says their container “works when I exec into it but not from my browser.” What are the two most likely causes?
- 2 You need to store an API key used by your app. Where does it go, and where must it never go?
- 3 Your final image is 1.4GB and mostly consists of build tools you no longer need at runtime. What technique addresses this directly?

Part 5 — Troubleshooting

- 1 A `docker compose up` fails with the web service repeatedly crashing on a database connection error, even though depends_on lists the db service. Diagnose and propose a fix.
- 2 A rebuild after a one-line code change takes just as long as the very first build. What Dockerfile ordering issue would you check first?

Part 6 — Final Project

Without referring back to Level 7, describe from memory the four main ingredients of a “production-style” Compose stack that this course covered, and why each one matters.

Final Exam — Answer Key

Part 1

- Docker packages an application with its exact environment (runtime, libraries, config) so it behaves identically across machines — solving “it works on my machine.”
- A container shares the host machine's OS kernel and only packages the application layer; a VM virtualizes an entire separate computer, including a full guest OS.
- An image is a static, read-only blueprint; a container is a live, running (or stopped) instance created from that blueprint. One image can produce many independent containers.
- A Dockerfile, when built, produces an image.

Part 2

- A container's default writable layer is ephemeral — it's deleted along with the container. A volume is Docker-managed storage that exists independently of any one container's lifecycle, so database data survives container removal and recreation.
- Docker Compose creates a private network per project and gives each service automatic name-based discovery on it. `localhost` inside a container refers to that container itself, not to any other container — so reaching another service requires its service name.
- A multi-stage build removes build-only tools (compilers, build-time dependencies) from the final shipped image, keeping only the finished artifacts. This produces a smaller, faster-to-deploy, lower-attack-surface image.

Part 3

(1) A minimal script Dockerfile needs only FROM, WORKDIR, COPY, and CMD — no RUN pip install is needed if there are no dependencies.

```
FROM python:3.12-slim
WORKDIR /app
COPY worker.py .
CMD ["python", "worker.py"]
```

(2) A FastAPI + Redis Compose file, publishing only the app's port:


```
services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - cache
  cache:
    image: redis:alpine
```

(3) The logs command:

```
docker logs -f api
```

Part 4

- Most likely causes: (a) the app is bound to 127.0.0.1 instead of 0.0.0.0 inside the container, or (b) the `-p` ports mapping was omitted or points at the wrong container-side port.
- It goes in a `.env` file, excluded from Git and from the Docker build context. It must never go in a Dockerfile (via `ENV/RUN` with the literal value) or be committed to version control.
- A multi-stage build — separating the build environment (with the heavy tools) from a minimal final runtime stage that only copies over the finished artifacts.

Part 5

- `depends_on` only waits for the db container to start, not for PostgreSQL inside it to be ready to accept connections. Fix: add a healthcheck to the db service and use the long-form `depends_on` with `condition: service_healthy` on the web service.
- Check whether dependency installation (`COPY requirements.txt + RUN pip install`) happens before `COPY . .` in the Dockerfile. If the whole app is copied first, every code change invalidates the cache for the dependency install layer too.

Part 6

- A multi-stage build — keeps the final image small and free of build-only tooling.
- A healthcheck with `condition: service_healthy` — ensures the app only starts once the database can truly accept connections, not just once its container exists.
- Secrets sourced from `.env` with a committed `.env.example` — keeps real credentials out of version control while documenting what's required.
- A named volume for persistent data — ensures the database survives container recreation.

Final “Can I Actually Do This?” Checklist

Be honest. This is for you, not for anyone else.

- I can explain what Docker is and why it exists, without notes.
- I can explain the difference between a container and a virtual machine.
- I can explain the difference between an image and a container.
- I can write a working Dockerfile for a simple Python/FastAPI app from memory.
- I know the core CLI commands to build, run, inspect, and clean up containers.
- I understand why containers need volumes for persistent data.
- I can write a docker-compose.yml connecting an app to a database.
- I understand why services use each other's names, not localhost, to connect.
- I know where secrets belong (and never belong).
- I understand what a multi-stage build removes and why that matters.
- I can debug a container that starts and immediately exits.
- I can debug a service that can't reach another service.
- I built and ran all three projects in this course myself.
- I know what I still need to practice (write it below).

What I still need to practice: _____

Complete Glossary

Term	Simple Definition	Example
Bind Mount	A direct link between a host folder and a folder inside a container.	-v \$(pwd):/app
Bridge Network	The default private network type Docker creates for containers to communicate.	The network Compose auto-creates per project.
Build Context	The set of files visible to Docker during a build.	The folder passed to docker build .
Compose Service	One container definition inside a docker-compose.yml.	The web or db service.
.dockerignore	A file excluding paths from the build context.	Excluding venv/ and .git/.
Dockerfile	A text file of instructions describing how to build an image.	FROM, COPY, RUN, CMD, etc.
Environment Variable	A configuration value passed into a running container from outside it.	DATABASE_URL, SECRET_KEY.
Ephemeral	Temporary; disappears when the container is removed.	A container's default writable layer.
Healthcheck	A Compose/Docker instruction that tests whether a service is truly ready, not just started.	pg_isready on a postgres service.
Hypervisor	Software that creates and manages virtual machines.	VirtualBox, Hyper-V.
Image	A static, read-only blueprint used to create containers.	python:3.12-slim + your app code.
Kernel	The core OS component that talks to hardware; shared by containers on the same host.	The Linux kernel shared by all your containers.
Layer	A cached step in an image's build, stacked to form the final image.	Each Dockerfile instruction is a layer.
Multi-Stage Build	A Dockerfile with multiple FROM stages, used to shrink the final image.	A builder stage + a minimal final stage.

Term	Simple Definition	Example
Port Mapping	Connecting a host port to a container's internal port.	-p 8000:8000
Registry	A storage/distribution service for Docker images.	Docker Hub.
Secret	Sensitive configuration that must never be hardcoded or committed to version control.	An API key or database password.
Service Discovery	Containers finding each other by name instead of IP address.	web reaching db:5432.
Tag	A label identifying a specific version of an image.	postgres:16
Volume	Docker-managed persistent storage independent of any single container's lifecycle.	pgdata volume for a database.

Cheat Sheet

Core Commands

```
docker build -t <name> .           # build an image
docker run -d -p 8000:8000 <name>  # run a container, detached, port mapped
docker ps -a                       # list all containers
docker images                      # list all images
docker logs -f <container>         # follow a container's logs
docker exec -it <container> bash   # open a shell inside a running container
docker stop <container>            # stop a container
docker rm <container>              # remove a stopped container
docker rmi <image>                 # remove an image
docker volume create <name>        # create a named volume
docker compose up -d               # start every service in the background
docker compose up --build          # rebuild images, then start
docker compose down                # stop and remove all services
docker compose down -v             # also removes volumes (careful!)
docker compose logs -f <service>   # follow one service's logs
docker compose exec <service> bash # shell into one running service
```

Minimal Dockerfile Pattern

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Minimal Compose Pattern

```
services:
  web:
    build: .
    ports: ["8000:8000"]
    depends_on: [db]
    environment:
      DATABASE_URL: postgresql://user:pass@db:5432/app
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

Rules of Thumb

- Copy dependency files and install them **before** copying the rest of your code (cache efficiency).
- Bind to 0.0.0.0, not 127.0.0.1, inside a container.
- Containers reach each other by **service name**, never localhost.
- Secrets go in .env (git-ignored), never in a Dockerfile.
- Databases always need a volume.
- depends_on controls start order, not readiness — use healthchecks for that.
- docker compose down -v deletes volumes — use plain `down` unless you mean it.

What To Learn Next

If You Enjoyed This Subject

- Docker networking in depth (custom bridge networks, overlay networks).
- Building and publishing your own images to Docker Hub.
- Docker security basics (running as a non-root user, image scanning).

If You Want To Become Job-Ready

- CI/CD pipelines that build and push Docker images automatically (GitHub Actions is a natural next step).
- Deploying a Docker Compose stack to a real cloud platform (Render, Railway, Fly.io, or an AWS/GCP VM).
- Basic Kubernetes concepts — the next step up once you're comfortable with multi-container apps, useful once a project grows beyond a single machine.

If You Want To Specialize

- Container orchestration (Kubernetes, Docker Swarm) for managing many containers across many machines.
- Platform/DevOps engineering — infrastructure-as-code (Terraform), observability, and deployment pipelines.
- Security-focused container work — image scanning, least-privilege containers, supply-chain security.

If You Want To Go Deeper

- How container isolation actually works under the hood (Linux namespaces and cgroups).
- Image layer internals and build cache optimization at an advanced level.
- Production healthcheck, logging, and observability patterns for containerized services.

◆ GOAL

Given where you already are — building FastAPI backends with PostgreSQL and moving toward Full Stack AI Engineering — the highest-leverage next step is usually: get one real project's Compose stack deployed to an actual cloud platform. That single exercise touches almost everything in this course at once, plus a first taste of real deployment.